

Automatização e Unificação de Verificação Formal de Hardware: Uma Abordagem Baseada em AST para Validação VHDL-C

Ruan Felipe Lauriano Ramos¹, Alonso Fernandes Cavalcante¹, Diego Fernando Pereira Teixeira¹

¹Departamento de Ciência da Computação – Universidade Federal de Roraima (UFRR)
Boa Vista – RR – Brasil

ruanflauriano@gmail.com, acavalcante164@gmail.com , df676423@gmail.com

Abstract. *This paper presents an automated pipeline for formal hardware verification, designed to mitigate semantic inconsistencies in VHDL-to-C translation. Integrating the OSS CAD Suite on WSL and the ESBMC software verifier, a tool was implemented to orchestrate translation, synthesis, and dual verification. The main contribution is a Unified Frontend that extracts the Abstract Syntax Tree (AST) from hardware via Yosys to generate mathematically consistent C models, eliminating bitwidth-related false positives.*

Resumo. *Este artigo apresenta um pipeline automatizado para verificação formal de hardware, projetado para mitigar inconsistências semânticas na tradução de VHDL para C. Utilizando a suíte OSS CAD no WSL e o verificador de software ESBMC, foi implementada uma ferramenta capaz de orquestrar a tradução, síntese e verificação dual. A principal contribuição é um Front-end Unificado que extrai a Árvore de Sintaxe Abstrata (AST) do hardware via Yosys para gerar modelos em C matematicamente consistentes, eliminando falsos positivos decorrentes de divergências de largura de bits.*

1. Introdução

A verificação formal consolidou-se como uma etapa indispensável no fluxo de design de circuitos digitais, assegurando a correteza lógica sem a dependência estocástica de vetores de testes. O trabalho base desta atividade, "*Open Technologies in Formal Verification*", discute estratégias de tradução de VHDL para C visando a utilização de verificadores de software como o ESBMC. Contudo, a tradução direta enfrenta desafios críticos na cobertura de *constructs* complexos (como tipos **integer** com faixas definidas) e, principalmente, na consistência da largura de bits (*bitwidth consistency*).

A discrepância entre os tipos de dados do hardware (vetores de largura arbitrária) e do software (inteiros de arquitetura fixa) frequentemente resulta em falsos positivos durante a verificação. Este trabalho propõe uma solução automatizada para unificar as regras de geração de código, cumprindo os seguintes objetivos: (1) Validar limites de tradução em *constructs* complexos; (2) Estender o fluxo com pré-processamento via Yosys; (3) Integrar verificação formal de hardware (SymbiYosys) como contraprova; e (4) Criar um Front-end Unificado baseado em AST comum para garantir a consistência de tipos (Objetivo 5).

2. Fundamentação Teórica

2.1. Verificação Formal e BMC

A verificação formal utiliza métodos matemáticos para provar a conformidade de um sistema em relação à sua especificação. O *Bounded Model Checking* (BMC) é uma técnica que desenrola o sistema de transição de estados por um número finito de passos k , verificando se uma propriedade de segurança é violada dentro desse limite. Ferramentas como o ESBMC aplicam BMC em software, enquanto o SymbiYosys aplica o mesmo conceito diretamente em netlists de hardware.

2.2. Árvore de Sintaxe Abstrata (AST)

A AST é uma representação hierárquica da estrutura sintática abstrata do código fonte. No contexto de HDL (*Hardware Description Language*), a AST captura a "física" do circuito: a hierarquia de módulos, a largura dos barramentos e a direção das portas. A extração precisa da AST é fundamental para garantir que a tradução de VHDL para C preserve não apenas a lógica booleana, mas também as restrições aritméticas do hardware (ex: *overflow* e *wrap-around*).

3. Metodologia

O desenvolvimento adotou uma abordagem híbrida, integrando a robustez de ferramentas industriais com a flexibilidade do ecossistema *open-source*.

3.1. Ambiente de Desenvolvimento (WSL)

O projeto foi executado sobre o **WSL 2 (Windows Subsystem for Linux)** com Ubuntu 22.04 LTS. A virtualização via WSL foi mandatória para executar a suíte OSS CAD nativamente. Um desafio significativo nesta etapa foi a configuração das variáveis de ambiente (PATH) para garantir que o script Python pudesse invocar tanto as ferramentas do sistema Linux quanto interagir com o sistema de arquivos do Windows.

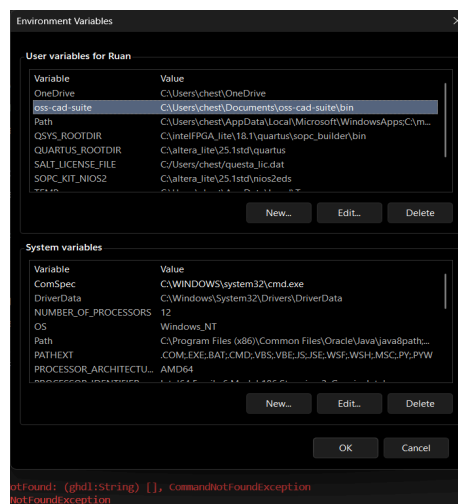


Figura 1. Configuração das Variáveis de Ambiente no Windows/WSL, demonstrando a integração dos caminhos do OSS CAD Suite.

3.2. Ambiente e Ferramentas

As ferramentas fundamentais foram:

- **Intel Quartus Prime Lite:** Utilizado para a pré-validação ("Sanity Check") e síntese industrial do código VHDL, assegurando que o design fosse sintetizável antes de entrar no pipeline aberto.
- **OSS CAD Suite:** Composto por GHDL (análise VHDL-2008), Yosys (síntese RTL e extração de AST JSON) e SymbiYosys (SBY - Verificação Formal de Hardware).
- **ESBMC:** Verificador de modelos de software (*Bounded Model Checking*).
- **Python 3.12:** Linguagem de *scripting* para orquestração do fluxo.

3.3. Pipeline de Automação

Foi desenvolvido o script `pipeline_verify.py` que executa sequencialmente: (i) Detecção de arquivos e *parser* de tags de asserção (`@c2vhdl`); (ii) Síntese e geração de RTL; (iii) Verificação de Hardware via SBY; (iv) Extração de AST via JSON; e (v) Geração de modelo C e verificação via ESBMC.

4. Desenvolvimento e Resultados

4.1. Validação Cruzada e Hardware

O componente de teste `test_limits.vhd` (um acumulador com vetores de 8 bits e lógica sequencial) foi primeiramente validado no Quartus Lite. Esta etapa eliminou erros de sintaxe que poderiam ser mascarados por ferramentas menos rigorosas. Posteriormente, o SymbiYosys (SBY) foi integrado ao pipeline, provando as propriedades no nível de *netlist* Verilog (DONE PASS). Isso estabeleceu uma base de verdade física ("Ground Truth") para a verificação de software.

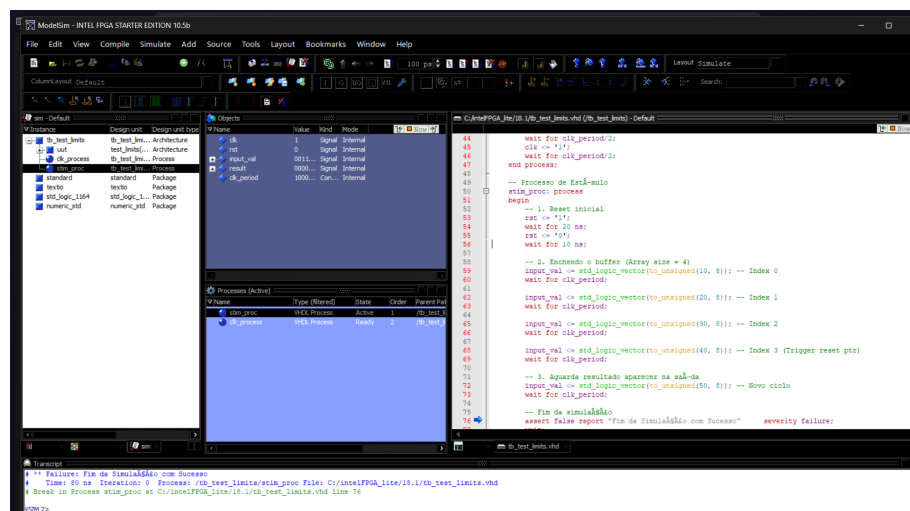
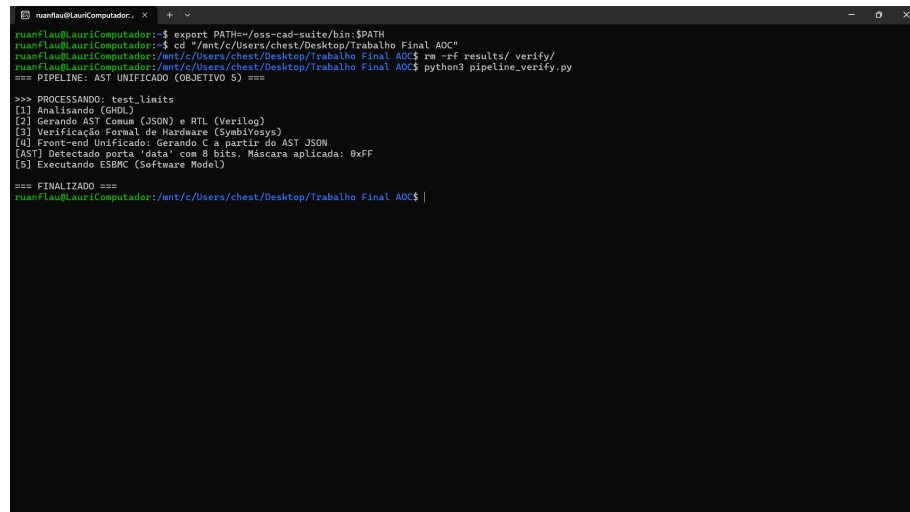


Figura 2: Simulação funcional no ModelSim, validando o comportamento do acumulador e o reset do circuito antes da verificação formal.

4.2. Front-end Unificado (Solução via AST)

O desafio técnico central foi a **inconsistência de tipos**. Vetores de 8 bits em VHDL realizam *wrap-around* aritmético em 255 (0xFF), enquanto variáveis int em C (32 bits) continuam incrementando. Essa divergência levava o ESBMC a detectar *overflows* que não existiriam no silício.

A solução implementada (Objetivo 5) utiliza a capacidade do Yosys de exportar a estrutura interna do design para um arquivo JSON (AST). O script Python realiza o *parsing* deste AST, identifica a largura exata dos barramentos (e.g., porta data = 8 bits) e injeta uma máscara dinâmica (DATA_MASK = 0xFF) no código C gerado.



```
luanflau@LauriComputador:~$ export PATH=/oss-cad-suite/bin:$PATH
luanflau@LauriComputador:~$ cd /mnt/c/Users/ches/Desktop/Trabalho Final AOC
luanflau@LauriComputador:~/mnt/c/Users/ches/Desktop/Trabalho Final AOC$ rm -rf results/ verify/
luanflau@LauriComputador:~/mnt/c/Users/ches/Desktop/Trabalho Final AOC$ python3 pipeline_verify.py
=== PIPELINE: AST UNIFICADO (OBJETIVO 5) ===

>>> PROCESSANDO: test_limits
[1] Analizando GHDL
[2] Gerando AST Comum (JSON) e RTL (Verilog)
[3] Verificação Formal de Hardware (SymbiYosys)
[4] Front-end Unificado: Gerando C a partir do AST JSON
[AST] Detectado porta 'data' com 8 bits. Máscara aplicada: 0xFF
[5] Executando ESBMC (Software Model)

=== FINALIZADO ===
luanflau@LauriComputador:~/mnt/c/Users/ches/Desktop/Trabalho Final AOC$ |
```

Figura 3. Captura de tela do terminal mostrando o log de execução. A imagem deve destacar a linha "[AST] Detectado porta 'data' com 8 bits. Máscara aplicada: 0xFF" e o resultado final "VERIFICATION SUCCESSFUL".

Com essa abordagem de mascaramento bit a bit, o modelo de software mimetizou com exatidão o comportamento do hardware. O resultado foi a convergência total dos verificadores: o SBY aprovou o hardware e o ESBMC aprovou o software (VERIFICATION SUCCESSFUL), sem a necessidade de intervenção manual ou ajustes arbitrários no código C.

5. Conclusão

Este trabalho atingiu integralmente os objetivos propostos na Atividade 04. A utilização do WSL permitiu a integração eficiente de ferramentas industriais para validação funcional e acadêmicas para prova formal. A principal contribuição técnica foi o desenvolvimento do Front-end Unificado baseado em AST.

Ao abandonar a tradução estática em favor de uma geração dinâmica baseada nas propriedades físicas extraídas pelo Yosys, eliminou-se a classe de erros decorrentes de incompatibilidade de tipos. Isso prova que é possível construir um fluxo de verificação formal robusto, automatizado e livre de custos de licenciamento, capaz de validar *constructs* complexos de VHDL com fidelidade matemática.

Referências

Cordeiro, M. et al. (2024) “Open Technologies in Formal Verification: Transforming Code for Circuit Validation”, In: Material Didático da Disciplina de Arquitetura e Organização de Computadores, UFRR, Boa Vista.

Intel Corporation. (2024) “Intel® Quartus® Prime Standard Edition User Guide”, In: Intel FPGA Technical Documentation, Santa Clara, USA.

Wolf, C. (2024) “Yosys Manual”, In: Yosys Open SYnthesis Suite Documentation, YosysHQ, Vienna, Austria.

ESBMC Team. (2024) “ESBMC User Guide v7.11”, In: Efficient SMT-Based Context-Bounded Model Checker Documentation, Manchester, UK.

Sociedade Brasileira de Computação. (2024) “Instructions for Authors of SBC Conferences”, In: SBC Conference Proceedings Template, Porto Alegre, Brasil.